

## Different Types of for Loops in Python

In Python, the for loop is used to repeat a block of code multiple times.

Basic syntax:

for variable in sequence:

```
# code
```

---

### 1. Simple for Loop with List

#### Code

```
fruits = ["Apple", "Mango", "Banana"]
```

```
for fruit in fruits:
```

```
    print(fruit)
```

#### Understanding

- fruits = list
- fruit = variable that stores one item at a time
- Loop runs 3 times

#### Output

```
Apple
```

```
Mango
```

```
Banana
```

---

### 2. for Loop with range()

Used for repeating code a fixed number of times.

---

#### (a) range(stop)

#### Code

```
for i in range(5):
```

```
    print(i)
```

#### Which is What?

- i = loop variable
- range(5) = numbers from 0 to 4

### Output

0  
1  
2  
3  
4

---

### (b) range(start, stop)

#### Code

```
for i in range(1, 6):  
    print(i)
```

#### Understanding

- Starts from 1
- Stops before 6

### Output

1  
2  
3  
4  
5

---

### (c) range(start, stop, step)

#### Code

```
for i in range(2, 11, 2):  
    print(i)
```

#### Which is What?

- 2 = start
- 11 = stop
- 2 = step/jump

### Output

2

4  
6  
8  
10

---

### 3. Nested for Loop

Loop inside another loop.

#### Code

```
for i in range(1, 4):  
  
    for j in range(1, 3):  
        print(i, j)
```

#### Which is What?

Outer Loop:

```
for i in range(1, 4)
```

Inner Loop:

```
for j in range(1, 3)
```

#### Understanding

- Outer loop runs 3 times
- Inner loop runs 2 times for each outer loop

#### Output

```
1 1  
1 2  
2 1  
2 2  
3 1  
3 2
```

---

### 4. for Loop with String

#### Code

```
name = "Python"
```

```
for ch in name:
```

```
    print(ch)
```

### **Understanding**

- Loop reads one character at a time

### **Output**

P

y

t

h

o

n

---

## **5. for Loop with Tuple**

### **Code**

```
numbers = (10, 20, 30)
```

```
for n in numbers:
```

```
    print(n)
```

### **Output**

10

20

30

---

## **6. for Loop with Dictionary**

### **(a) Print Keys**

#### **Code**

```
student = {  
    "name": "Rahul",  
    "age": 21
```

```
}
```

```
for key in student:
```

```
    print(key)
```

### **Output**

name

age

---

### **(b) Print Values**

#### **Code**

```
student = {  
    "name": "Rahul",  
    "age": 21  
}
```

```
for value in student.values():
```

```
    print(value)
```

### **Output**

Rahul

21

---

### **(c) Print Key and Value Both**

#### **Code**

```
student = {  
    "name": "Rahul",  
    "age": 21  
}
```

```
for key, value in student.items():
```

```
    print(key, value)
```

### **Which is What?**

- key = dictionary key
- value = dictionary value
- items() = returns both

### **Output**

name Rahul

age 21

---

### **7. for Loop with break**

Stops the loop completely.

#### **Code**

```
for i in range(1, 6):
```

```
    if i == 3:
```

```
        break
```

```
    print(i)
```

#### **Understanding**

- Loop stops when i = 3

### **Output**

1

2

---

### **8. for Loop with continue**

Skips current iteration.

#### **Code**

```
for i in range(1, 6):
```

```
    if i == 3:
```

```
        continue
```

```
    print(i)
```

### Output

1  
2  
4  
5

---

### 9. for Loop with else

else runs if loop finishes normally.

#### Code

```
for i in range(3):  
    print(i)  
  
else:  
    print("Loop Finished")
```

### Output

0  
1  
2  
Loop Finished

---

### 10. Infinite Style Loop Using for

#### Code

```
for i in iter(int, 1):  
    print("Hello")
```

#### Understanding

- Rarely used
  - Creates an endless loop
- 

### Most Important for Exams

Usually asked in exams:

1. Simple for loop

2. range() loop
3. Nested loop
4. Loop with break
5. Loop with continue
6. Loop with list/string
7. Dictionary loop

---

### Very Short Notes (Quick Revision)

| Type            | Use                |
|-----------------|--------------------|
| Simple loop     | Iterate items      |
| range()         | Repeat fixed times |
| Nested loop     | Loop inside loop   |
| break           | Stop loop          |
| continue        | Skip iteration     |
| else            | Runs after loop    |
| Dictionary loop | Access keys/values |

## while Loop in Python

A while loop repeats code **as long as the condition is True**.

---

### Basic Syntax

while condition:

    # code

---

### Flow of while Loop

1. Check condition
  2. If condition is True → execute code
  3. Again check condition
  4. Repeat until condition becomes False
- 

### 1. Simple while Loop

#### Code

i = 1

while i <= 5:

    print(i)

    i = i + 1

---

#### Which is What?

i = 1

- Initialization
  - Starting value of loop
- 

while i <= 5:

- Condition
  - Loop runs while condition is True
- 

i = i + 1

- Increment
  - Increases value of i
- 

### **Output**

1  
2  
3  
4  
5

---

## **2. Infinite while Loop**

Condition always remains True.

### **Code**

```
while True:  
    print("Hello")
```

### **Understanding**

- True is always true
  - Loop never stops automatically
- 

## **3. while Loop with break**

break stops the loop immediately.

### **Code**

```
i = 1  
  
while i <= 5:  
  
    if i == 3:  
        break  
  
    print(i)  
    i = i + 1
```

---

### Understanding

When  $i = 3$

`break`

stops the loop.

---

### Output

1

2

---

### 4. while Loop with continue

`continue` skips current iteration.

### Code

```
i = 0
```

```
while i < 5:
```

```
    i = i + 1
```

```
    if i == 3:
```

```
        continue
```

```
    print(i)
```

---

### Understanding

When  $i = 3$

`continue`

skips printing.

---

### Output

1

2

4

5

---

### 5. while Loop with else

else runs when loop ends normally.

#### Code

```
i = 1
```

```
while i <= 3:
```

```
    print(i)
```

```
    i = i + 1
```

```
else:
```

```
    print("Loop Finished")
```

---

#### Output

1

2

3

Loop Finished

---

### 6. Nested while Loop

while loop inside another while loop.

#### Code

```
i = 1
```

```
while i <= 3:
```

```
    j = 1
```

```
while j <= 2:
```

```
    print(i, j)
```

```
    j = j + 1
```

```
i = i + 1
```

---

### Which is What?

Outer loop:

```
while i <= 3
```

Inner loop:

```
while j <= 2
```

---

### Output

```
1 1
```

```
1 2
```

```
2 1
```

```
2 2
```

```
3 1
```

```
3 2
```

---

## 7. while Loop for User Input

### Code

```
password = ""
```

```
while password != "python":
```

```
    password = input("Enter Password: ")
```

```
print("Correct Password")
```

---

### Understanding

- Loop keeps asking until correct password entered

---

## 8. Reverse Counting Using while

### Code

```
i = 5

while i >= 1:
    print(i)
    i = i - 1
```

---

### Output

```
5
4
3
2
1
```

---

## 9. Sum of Numbers Using while

### Code

```
i = 1
sum = 0

while i <= 5:
    sum = sum + i
    i = i + 1

print(sum)
```

---

### Understanding

```
sum = sum + i
```

adds numbers one by one.

---

## Output

15

---

### Difference Between for and while

#### for Loop

Used when repetitions are known

Easier syntax

Mostly used with sequence

#### while Loop

Used when repetitions are unknown

Condition-based

Mostly used with conditions

---

### Very Important Exam Points

#### Three Important Parts of while

1. Initialization

`i = 1`

2. Condition

`while i <= 5`

3. Increment/Decrement

`i = i + 1`

Without increment/decrement → infinite loop may happen.

---

### Quick Revision Table

| Type           | Purpose                      |
|----------------|------------------------------|
| Simple while   | Repeat until condition false |
| Infinite while | Never-ending loop            |
| break          | Stop loop                    |
| continue       | Skip iteration               |
| else           | Executes after loop          |
| Nested while   | Loop inside loop             |